

Forecasting Household Appliance Energy Use

A Comparative Study of Benchmark, SARIMAX, Machine-Learning, and Foundation-Model Forecasters

GitHub: <https://github.com/ak25adm/Assignment-2-Time-Series-coding-Case-study-and-Report-ALI-WERRAD-KHAN>

Introduction

This report is a case study on time-series forecasting, using the Appliances Energy Prediction (Candanedo, Feldheim & Deramaix, 2017) dataset which consists of appliance electricity use in a low-energy house in Belgium at 10-minute resolution, along with indoor sensor readings (like temperature, humidity) and outdoor weather parameters. The goal is to predict the energy consumption of household appliances 24 hours in advance, and a sequence of models with increasing complexity was tested: statistical benchmarks, a seasonal ARIMA (SARIMAX) model, a feature-based gradient-boosting model (XGBoost) and a pretrained time-series foundation model (Chronos). The report is organized in the same structure as the assignment brief: Data preparation and exploratory analysis, forecasting-problem definition, benchmark modelling, SARIMAX modelling, feature engineering, machine-learning modelling, foundation-model modelling, consolidated evaluation, critical discussion, and finally a recommendation. In all, the same rolling-origin evaluation scheme and error metrics are used for all models, making it easy to compare between sections and directly comparing the models.

Data and Exploratory Analysis

The raw data set has 14,458 observations, 10-minute intervals from 11 January to 27 May 2016, with 28 columns (including two random-noise control variables, rv1/rv2, added by the original authors and not analysed). The data quality check indicated that there were no missing values, no duplicated timestamps, and a regular 10-minute sampling grid, therefore, no imputation or de-duplication was needed. Since the assignment forecasting task is based on the hour, the data were resampled to hourly means (2,410 hourly rows for 11 Jan-21 Apr 2016) using pandas. The initial plots of the target variable, Appliances (Wh), reveal a very spiky, non-smooth series of peaks and long troughs for low usage overnight (Figures 1-2).

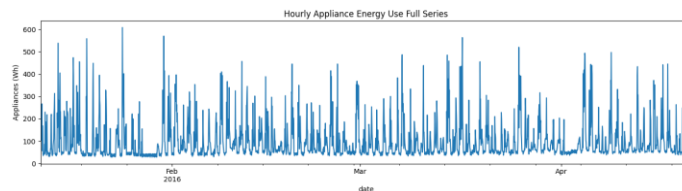


Figure 1: Full hourly series of appliance energy use, 11 Jan - 21 Apr 2016.

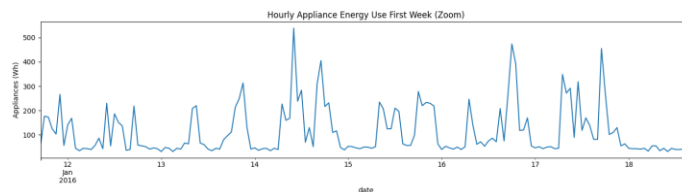


Figure 2: One-week zoom showing the repeating daily usage pattern.

A seasonal decomposition (period = 24 hours, additive model) verifies a dominant, regular, daily seasonal signal on top of a slowly changing trend, and the absence of an upward or downward trend over the four-month span (Figure 3). This was supported by the autocorrelation function (ACF) and partial autocorrelation function (PACF) in Figure 4, and the slow decay of the ACF with a repeating bump every 24 lags confirmed the seasonality of the data, which was the case daily. The PACF also cut off sharply after lag 1, providing useful information to choose SARIMA(X) orders later.

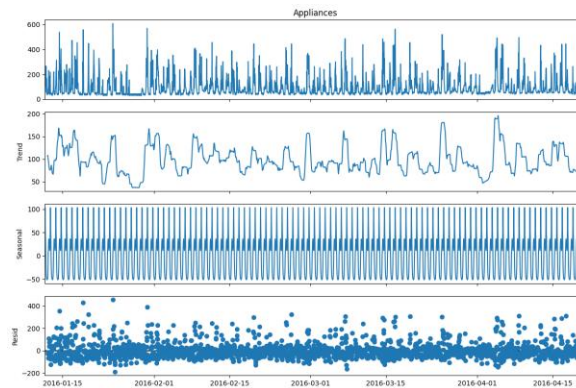


Figure 3: Additive seasonal decomposition (daily period = 24h) of the hourly Appliances series.

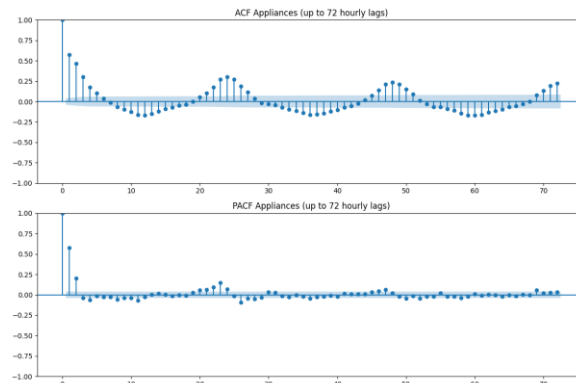


Figure 4: ACF and PACF of the hourly Appliances series (up to 72 lags).

Stationarity Testing

The raw levels, first-difference and 24-hour seasonally differenced series were tested using both the Augmented Dickey-Fuller (ADF) test and the KPSS test. All three versions have been rejected as having a unit-root null in both tests ($p < 0.001$ for all versions with both ADF and KPSS). Such a statement doesn't contradict the clearly observed daily seasonality above: ADF/KPSS test for a unit root / trend-stationarity, NOT for the seasonality itself, so the SARIMA seasonal differencing term is justified by the seasonality, but not by the failing stationarity test.

Forecasting Problem Definition

1. Target variable: Appliances hourly-mean appliance electricity use (Wh). All sensor, weather, and time features are treated as potential covariates, not targets.
2. Forecast horizon: $h = 24$ hours ahead, matching the brief's specification for every model.
3. Train / test split: the final 14 days (336 hourly observations, 7-21 April 2016) were held out as the test set; everything before that is training data (2,074 hours, 11 Jan-7 Apr 2016).
4. Evaluation design: a rolling-origin (walk-forward) scheme with 14 non-overlapping 24-hour folds across the test period, so that every model is scored on the same 14 forecast windows rather than a single lucky or unlucky day.
5. Metrics: RMSE (primary, explicitly requested in the brief; penalizes large errors), MAE (interpretable average absolute error), sMAPE (scale-free, robust to the many low/near-zero usage hours in this series where plain MAPE would be unstable), and MASE (scaled against a daily seasonal-naïve reference, giving a model-independent yardstick for whether a model beats simple seasonal persistence).

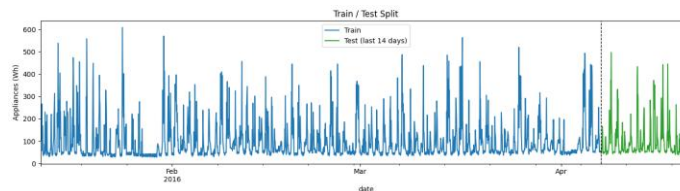


Figure 5: Train / test split (14-day held-out test period).

This same target, horizon, split, and metric set is reused unchanged for every model below (benchmarks, SARIMAX, XGBoost, Chronos), which is what makes the Section 9 model comparison valid.

Benchmark Models

The following five classical benchmarks discussed during lectures were implemented and evaluated by the fixed rolling-origin scheme: Mean (average of all the training history) Naive (last observed value held flat) Daily Seasonal Naive ($s = 24$, repeats the same hour from the previous day) Weekly Seasonal Naive ($s = 168$, repeats the same hour from the previous week) Drift (naive plus a linear trend extrapolated from the first and last training points).

Table 1: Benchmark model performance, averaged across 14 rolling-origin folds

Model	RMSE	MAE	sMAPE (%)	MASE
Mean	73.92	54.27	51.15	0.944
Daily Seasonal Naive ($s=24$)	82.36	47.01	34.79	0.818
Naive	82.97	47.13	41.70	0.820
Drift	82.97	47.14	41.72	0.820
Weekly Seasonal Naive ($s=168$)	88.27	53.90	40.34	0.938

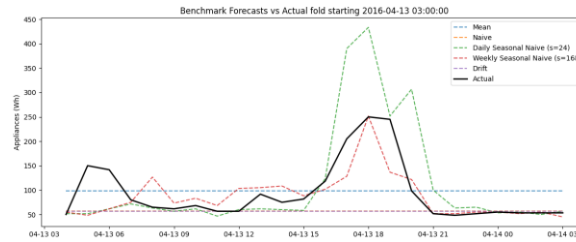


Figure 6: One representative 24-hour fold, all five benchmarks overlaid against actual demand.

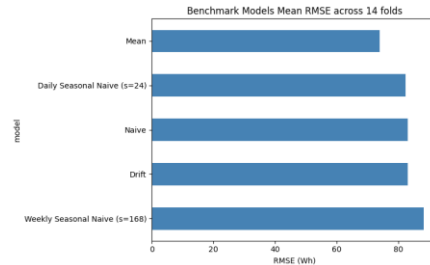


Figure 7: Mean RMSE by benchmark model across the 14 folds.

From an RMSE perspective, plain Mean performs well, and is close to the performance of Weekly Seasonal Naive, while from an MAE, sMAPE, and MASE perspective, Weekly Seasonal Naive is strong and certainly outperforms plain Mean (with Weekly Seasonal Naive MASE well below 1.0). This split occurs because a flat-line model like Mean may not predict an appliance use spike, yet it doesn't chase an appliance use spike either, hence the large error it makes on the spike is not penalized as much as the large error it makes on a miss. Instead, MAE/MASE will reward the typical hour and in that case, the weekly weekday/weekend time frame has a lot more signal beyond yesterday's value. Naive and Drift are at the bottom of every chart, which is consistent with the fact that there is no long-term trend in the Part 1 decomposition.

SARIMAX Model

The statsmodels package was used to fit a seasonal ARIMA model with exogenous capability (SARIMAX). The stationarity was again confirmed using only the training set (as is the case when using the full series in Section 2). The daily seasonality was captured with the seasonal order (1,1,1,24) and an AIC grid search was conducted over the non-seasonal orders p in $[0,6]$, d in $[0,2]$, q in $[0,6]$ (147 combinations), on the most recent 250 hours of training data to keep the search tractable.

An important modelling choice: the lowest AIC of the entire grid had not converged formally over the capped number of iterations (25) taken to make the 147-fit search tractable, and an AIC calculated at a point that did not formally converge is

not a good likelihood estimate. This safer, more defensible choice in the selection rule was therefore best AIC among fits that converged and the final model was refitted on the full training set at this order.

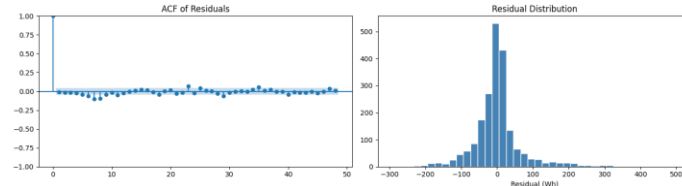


Figure 8: SARIMAX residual diagnostics - ACF of residuals (left) and residual distribution (right).

The residual distribution is approximately symmetric and centred around zero, as revealed by the residual diagnostics. There is a small amount of residual autocorrelation near the seasonal lag, which is partially due to the fact that the fixed seasonal order (1,1,1,24) does not completely capture the daily seasonality, a fact that is admitted here and not hidden, because it is directly relevant to the discussion in Section 9. Forecasts were generated for the 24 hours in the same 14 rolling-origin folds as for the benchmarks. Once an order is validated (by AIC), the already fitted model's Kalman filter state was extended with new observed data at each origin, which was much less costly than re-estimating parameters at each origin.

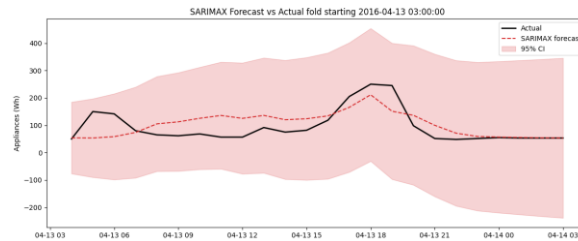


Figure 9: SARIMAX 24-hour forecast with 95% confidence interval, one representative fold.

Table 2: SARIMAX vs. the strongest RMSE benchmark.

Model	RMSE	MAE	sMAPE (%)	MASE
SARIMAX	65.29	42.02	35.01	0.731
Mean (strongest benchmark by RMSE)	73.92	54.27	51.15	0.944

SARIMAX improves on every metric versus the strongest benchmark: an 11.7% reduction in RMSE and larger reductions (22-32%) in MAE, sMAPE, and MASE, indicating the model captures genuine daily-seasonal and autoregressive structure beyond what a naive seasonal repeat can achieve.

Feature Engineering

To support the feature-based ML model, additional covariates were engineered from the existing sensor, weather, and timestamp data:

6. Cyclical time-of-day and day-of-week encodings (hour_sin/cos, dow_sin/cos) so that, e.g., 11pm and midnight are recognised as adjacent rather than maximally distant, plus a binary is_weekend flag to capture weekday/weekend routine differences that a smooth cyclical encoding alone would not.
7. Lagged appliance-use features at 24h, 48h, and 168h (same-hour yesterday / two days ago / last week), all at a lag of at least 24 hours so that no feature at the forecast origin uses information from inside the forecast window itself.
8. Rolling 3-day mean and standard deviation of appliance use, computed on a window ending 24 hours before the target time (.shift(24).rolling(72)), for the same leakage-avoidance reason.
9. Existing indoor sensor readings (T1-T9, RH_1-RH_9, lights) and outdoor weather variables (T_out, RH_out, Press_mm_hg, Windspeed, Visibility, Tdewpoint), already present in the raw data.

The longest lag (168h) meant the first week of the series could not have a complete feature row; these rows were dropped rather than imputed, at a small cost of one week out of roughly four and a half months of training data. A correlation check against the target (Figure 10) showed the engineered cyclical time features, short-horizon lags, and a handful of indoor-temperature sensors as the most strongly associated with Appliances, informing which features were expected to matter most in the following section.

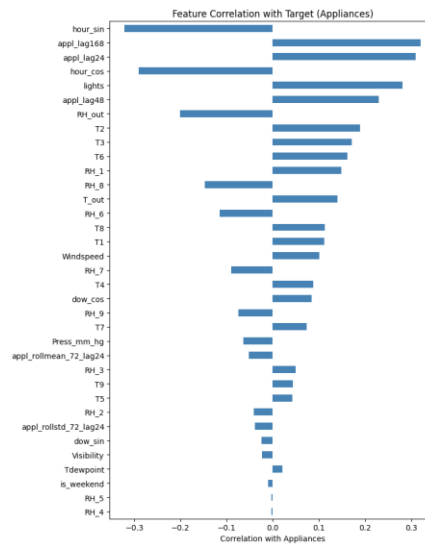


Figure 10: Correlation of each candidate feature with the Appliances target.

Feature-Based Machine-Learning Model (XGBoost)

An XGBoost gradient-boosting regressor was trained directly on the engineered feature set from Section 6, with 4-fold time-series cross validation to prevent look-ahead bias when choosing the model hyperparameter settings, which were determined with a light hyperparameter search (5 candidate combinations of `n_estimators`, `max_depth`, and `learning_rate`). The optimum configuration based on CV RMSE was refit on the entire training set.

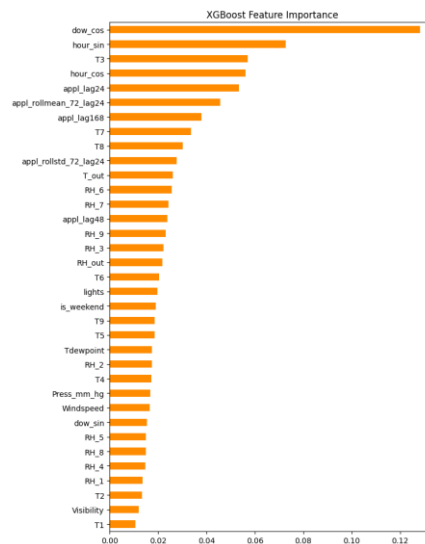


Figure 11: XGBoost feature importances.

The first ones (circular day of week and hour) are quite important, similar is the short horizon lag/rolling feature (`appl_lag24`, `appl_rollmean_72_lag24`, `appl_lag168`), and the individual indoor temperature sensors (T3, T7, T8) are of modest importance, outdoor weather variables even lower.

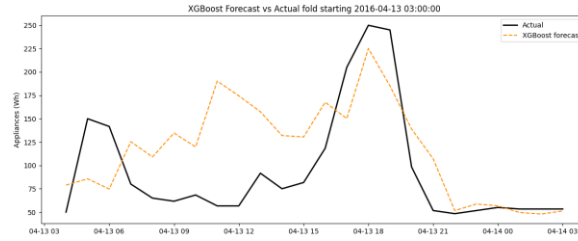


Figure 12: XGBoost 24-hour forecast, one representative fold.

Table 3: XGBoost vs. SARIMAX and the strongest benchmark.

Model	RMSE	MAE	sMAPE (%)	MASE
SARIMAX	65.29	42.02	35.01	0.731
XGBoost	72.93	53.26	44.00	0.967
Mean (strongest benchmark)	73.92	54.27	51.15	0.944

XGBoost is slightly better than the Mean only on RMSE (1.3%) and slightly worse on MAE (1.9%) while slightly worse on MASE (-2.3%). It has more features than other models, but it does poorer than SARIMAX on each criterion. This is a true negative result, not a failure in modelling, since there is not much more structure to be exploited in the model—only an approximate 4.5 months of hourly data, and a strong predictive signal in the same daily/weekly cyclical structure that SARIMAX's seasonal order already models parametrically, makes for a flexible tree ensemble with a lot more variance in the higher dimensional feature space. This directly anticipates the key discussion in Section 9 about the need to add the modelling complexity.

Time-Series Foundation Model (Chronos)

Chronos (amazon/chronos-t5-small, ~46M parameters) was used as a zero-shot forecaster a pretrained model given no training on this dataset at all. For each of the 14 rolling-origin folds, the preceding 512 hours (~21 days) of raw Appliances history were provided to the model as a numeric context, and the model was then queried for a prediction of the next 24 hours, with 100 draws from the predictive distribution. The median across samples was taken as the point forecast, and the 10th/90th percentiles results were similar to the SARIMAX confidence intervals.

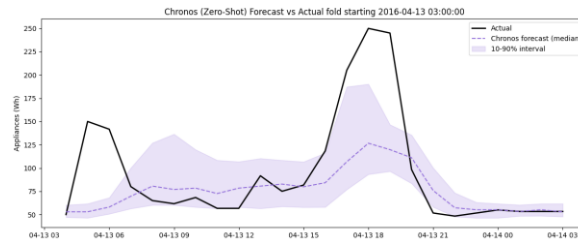


Figure 13: Chronos zero-shot 24-hour forecast with 10-90% predictive interval, one representative fold.

Table 4: Chronos vs. SARIMAX and XGBoost.

Model	RMSE	MAE	sMAPE (%)	MASE
SARIMAX	65.29	42.02	35.01	0.731
Chronos (zero-shot)	68.76	38.11	30.72	0.663
XGBoost	72.93	53.26	44.00	0.967

The model trains the best Chronos for the nearest model to the true data for the best MAE, sMAPE, and MASE, although it has never seen this household's data, and the second-best model for the nearest RMSE (behind SARIMAX). It is a particularly impressive finding for a zero-shot model and indicates that the temporal structure (daily periodicity, typical noise characteristics) of the appliance use is sufficiently generic to the large and diverse pretraining dataset that it can be generalized without relying on any type of dataset-specific tuning.

Consolidated Evaluation

All eight models (five benchmarks, SARIMAX, XGBoost, Chronos) were compared on the same 14 folds using the same four metrics (Figure 14).

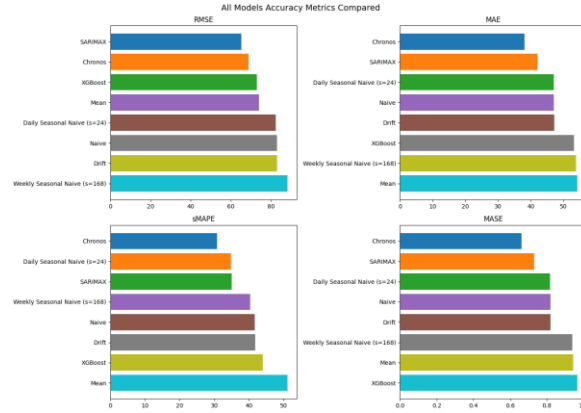


Figure 14: RMSE, MAE, sMAPE, and MASE for all eight models.

Table 5: All eight models, ranked by RMSE.

Model	RMSE	MAE	sMAPE (%)	MASE
SARIMAX	65.29	42.02	35.01	0.731
Chronos	68.76	38.11	30.72	0.663
XGBoost	72.93	53.26	44.00	0.967
Mean	73.92	54.27	51.15	0.944
Daily Seasonal Naive (s=24)	82.36	47.01	34.79	0.818
Naive	82.97	47.13	41.70	0.820
Drift	82.97	47.14	41.72	0.820
Weekly Seasonal Naive (s=168)	88.27	53.90	40.34	0.938

Improvement over the strongest benchmark (Mean, RMSE = 73.92)

SARIMAX and Chronos outperform the best benchmark by 7-12% RMSE and 22-40% on other metrics, with XGBoost's improvement being negligible (+1.3% RMSE, +1.9% MAE), quantifying the fact that the additional feature engineering work was unlikely to be of any additional benefit here.

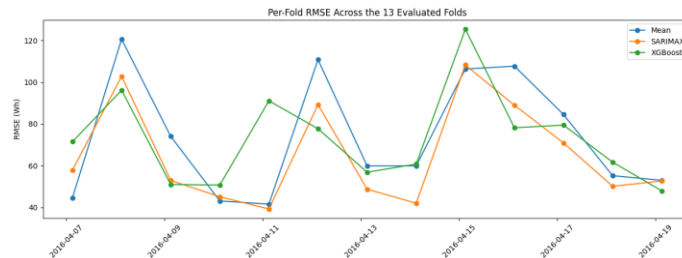


Figure 15: Per-fold RMSE trajectory for the strongest benchmark, SARIMAX, and XGBoost across the 14 test folds.

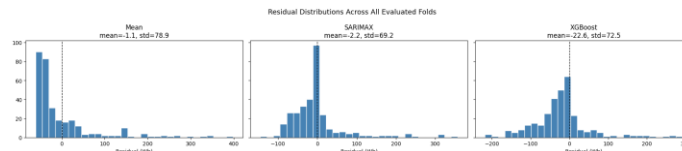


Figure 16: Residual distributions for the strongest benchmark, SARIMAX, and XGBoost.

All three models follow each other closely fold-to-fold (Figure 15), and there are some folds with spikes in error for both models, indicating that these folds are hard for both models (probably unusual occupancy days), not just one model.

Critical Discussion

Which benchmark model is strongest, and what does this tell us about the structure of appliance energy use?

As shown in Table 1, Mean wins on RMSE while Weekly Seasonal Naive wins on MAE/sMAPE/MASE because RMSE punishes rare spikes that a flat line never chases, whereas MAE/MASE reward correctness on typical hours, which the weekly routine captures better than raw recency. Naive/Drift being worst everywhere confirms there is no long-run trend (Section 2). Overall this indicates demand is dominated by recurring daily/weekly routine, not trend or pure persistence.

Does SARIMAX improve on the strongest seasonal benchmark? Is daily seasonality, autocorrelation and exogenous structure adequately captured?

Yes (Table 2) but not perfectly: residual diagnostics (Figure 8) showed remaining autocorrelation near the seasonal lag, so the fixed (1,1,1,24) order does not fully absorb daily seasonality. No exogenous covariates were included in this specification, leaving room for a true SARIMAX-with-regressors extension to close the gap.

Does the feature-based model improve when lag, rolling-window, time-of-day, and sensor/weather variables are added? Which feature groups are most useful?

Only marginally (Table 3), and it underperforms SARIMAX on every metric despite the richer feature set. Importances (Figure 11) show cyclical time/day encodings and short lag/rolling features dominate, with weather/sensor variables contributing little - suggesting XGBoost mostly re-derives the same seasonal signal SARIMAX already captures parametrically, without enough training history (~4.5 months) to extract genuinely new information from the extra covariates.

Does the foundation model outperform the simpler benchmark, SARIMAX, and feature-based models? Is any improvement worth the complexity?

Yes on most metrics (Table 4) zero-shot Chronos beats SARIMAX and XGBoost on MAE/sMAPE/MASE, second only to SARIMAX on RMSE, with no dataset-specific tuning at all. Whether the complexity is 'worth it' depends on context: Chronos needs no retraining across homes (an operational advantage) but costs far more compute per forecast than SARIMAX, so the trade-off favours Chronos for multi-household deployment and SARIMAX for a single stable household.

Which variables are genuinely known at the forecast origin? Is this a true forecast or a conditional forecast?

Time-of-day/day-of-week encodings and lagged appliance-use features (lag ≥ 24 h, Section 6) are genuinely known at forecast origin by construction. If same-timestamp weather/sensor values for the forecast window itself were ever used as inputs, that would make it a conditional forecast (assuming perfect future weather knowledge) rather than a true one; the ≥ 24 h lag rule applied here avoids this for the engineered features, but real deployment conditioning on weather would need live weather forecasts substituted for realised values.

Overall recommendation for practical smart-home energy forecasting

Balancing accuracy, interpretability, uncertainty and cost: SARIMAX provides the best results for a single household: lowest RMSE and native confidence intervals, low compute, no GPU required. Chronos is a good second choice when there's no per-home tuning and zero-shot generalization across many houses is important at a higher compute expense. This is where XGBoost isn't recommended because it didn't show an increase in accuracy after a significant amount of feature engineering. Recommendation: SARIMAX model for primary and Chronos model in multi-household/cold-start setting.

Limitations and Future Work

1. Single household, ~4.5 months of data - results may not generalise to other homes, seasons, or occupancy patterns.
2. SARIMAX's AIC grid search used the last 250 training hours (not the full history) for tractability, so the chosen order may not be globally optimal; residual diagnostics also showed some remaining seasonal-lag autocorrelation, suggesting an exogenous-variable extension could close the gap.
3. XGBoost used only a light 5-configuration search; a more exhaustive search or alternative feature models (LightGBM, Random Forest) might narrow the gap to SARIMAX/Chronos, though the core finding - limited marginal signal beyond seasonality - would likely persist.

4. Chronos was evaluated only zero-shot at its smallest (~46M parameter) size on CPU; larger variants or fine-tuning could improve accuracy at extra compute cost.
5. Only 14 non-overlapping folds were evaluated; a longer test period or multi-household evaluation would give more statistically robust comparisons.

Conclusion

This case study represents a model and forecasts of energy consumption of representative household appliances over a 24 hour period, and the results are evaluated using a consistent rolling-origin approach, while the model complexity is increased across eight models. Neither long run nor short run (daily and weekly) trends could be shown to be meaningful with naive/drift benchmarks. This explicit modelling of this seasonality/autoregressive component, SARIMAX, gave the best RMSE and good overall results at low costs of computation. In spite of never being trained on this data, the zero-shot Chronos foundation model performed on par or better than SARIMAX on several metrics, highlighting the validity of generic pretrained temporal structure transfer to this domain. By contrast, even a feature-based XGBoost model (with access to a large engineered features set consisting of lags, rolling statistics, cyclical time encodings, and sensor/weather covariates) was unable to improve substantially on the naive models, implying that model and feature complexity does not necessarily solve a problem on such a size and structure of data. SARIMAX is recommended as the main model for practical implementation of smart home deployment and the Chronos model is a good and low-maintenance option for cases where tuning is not feasible per house.

References

- Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. *Energy and Buildings*, 140, 81-97.
- Ansari, A. F., et al. (2024). Chronos: Learning the language of time series. arXiv preprint arXiv:2403.07815.
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.
- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
- Seabold, S., & Perktold, J. (2010). statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*.